

Team Guide

Who does what, how we work, and how to check progress

Learnership Management System - written in simple English

PM

BE-1

BE-2

BE-3

FE

QA

Part A - For the whole team: the big picture and how to work

Part B - Who starts first and what each person builds

Part C - For the lead: how to manage and check progress

Contents

A1. Start Here: The Big Picture	4
What are we building?	4
Who uses it?	4
The 6 pieces (we call them apps)	4
The 5 golden rules	4
A2. Words We Use, in Simple English	5
How one click works: a student opens the home page	5
Words you will hear every day	5
A3. How to Do a Task, Step by Step	6
The Git commands (copy and use)	6
Do and Do not	6
A4. How We Talk to Each Other	7
Daily standup message (copy this)	7
How to ask a good question	7
B1. The Team and Who Owns What	8
If you have to change the roles	8
Work by person and sprint (task numbers)	8
B2. What Each Person Does First	9
Week 1 (Sprint 0): day by day	9
Weeks 2 and 3 (Sprint 1): what to do while others finish	9
B3. Sprint by Sprint: Who Builds What	10
C1. Your Weekly Rhythm as the Lead	12
Weekly summary message (post every Friday)	12
Setting up the tracker (once)	12
C2. How to Check Progress Fairly	13
Traffic light (also shown in the tracker)	13
Personal scorecard (fill in at the end of each sprint)	13
C3. Reviewing Pull Requests	14
Good review comment examples	14
C4. When Someone Is Behind or Stuck	15
Step by step	15
Common causes and what to do	15
When the whole project is late	15
C5. Sprint Gates and Templates	16
Pull Request template	16

Bug report template (QA posts in #lms-qa)	16
Severity levels	16
Friday retro: three questions	16
C6. Your First Week Checklist	17

A1. Start Here: The Big Picture

Read this page first. Everyone on the team must understand it before writing any code.

What are we building?

We are building a website for our company. Students can look at our learnerships and courses and apply for them. Facilitators (teachers) run the courses. The admin controls everything. Think of it like a small school on the internet.

Who uses it?

Person	What they can do in simple words
Student	Looks at courses on the home page. Applies. After approval, sees their courses, test marks, feedback and attendance.
Facilitator	Teaches units. Writes test marks and feedback. Takes attendance. If the admin allows it, can also add, change and delete courses.
Admin	Can do almost everything. Only the admin can give a facilitator the right to manage courses.

The 6 pieces (we call them apps)

App	Simple meaning	Owner
accounts	People: who you are, login, roles	BE-1
courses	Courses and the units inside them	BE-2 and BE-3
enrollments	Applying and being accepted	BE-3 and BE-1
assessments	Tests, marks and feedback	BE-3 and BE-2
attendance	Class sessions and who came	BE-1
core + templates	Home page, dashboards, and how every page looks	FE with BE-3

The 5 golden rules

1	Never work directly on main or develop . Always make your own branch for each task.
2	Every change goes through a Pull Request . The lead must look at it before it is merged.
3	If you are stuck for 30 minutes , ask for help in Slack. Waiting quietly costs the team more than asking.
4	Every page that needs a permission must have a test . A hidden button is not security.
5	Write your standup message every day , even if you have nothing to report. It helps the team trust each other.

A2. Words We Use, in Simple English

Django is like a restaurant. A customer (the browser) asks for a dish (a page). The waiter (URL) takes the order to the right cook (view). The cook gets ingredients from the store room (model and database) and puts the food on a plate (template). Then the waiter brings it back.

How one click works: a student opens the home page

Step	What happens	Restaurant picture
1	The student types our website address.	Browser
2	Django reads the address and finds the matching line in <code>urls.py</code> .	URL = the waiter
3	That line points to <code>HomeView</code> . The view is a small piece of Python that decides what to do.	View = the cook
4	The view asks the database: 'give me all courses that are Published'.	Model = the store room
5	The view gives the courses to <code>home.html</code> , which is the plate design.	Template = the plate
6	The finished page is sent back to the student's screen.	Response

Words you will hear every day

Word	Simple meaning	Example in our project
Model	A table in the database, written as Python code	Course, Unit, Application
Migration	A file that tells the database to change (add a table or column)	Adding the 'can_manage_courses' field
View	Python code that answers one web request	CourseCreateView
URL	The web address that reaches a view	<code>/courses/create/</code>
Template	An HTML file with holes that Django fills with data	<code>course_detail.html</code>
Form	Checks the data a person types before we save it	CourseForm checks that end date is after start date
Service	A function that holds a business rule, so views stay small	<code>approve_application()</code>
Mixin	A small helper class that adds a rule to a view	CourseManagerRequiredMixin
Queryset	A list of database rows you asked for	Only the units this facilitator teaches
Foreign key	A link from one table to another	Unit points to Course
Branch	Your own copy of the code where you work safely	<code>feature/LMS-203-course-crud</code>
Pull Request (PR)	A request to add your branch to the main code, so others can check it	PR titled 'LMS-203: Course create view'
Merge	Joining your checked work into develop	After the lead approves
Test	Code that checks other code still works	A student must get 403 on <code>/courses/create/</code>
Blocker	Something that stops you from continuing	Waiting for the User model to be merged

A3. How to Do a Task, Step by Step

Follow these 11 steps for **every** task. It looks long, but after three tasks it becomes a habit.

1	Pick the first task in your list that is not blocked. Look at the 'Depends on' column: the other task must be Done (merged) first.
2	Post in the task thread: 'Starting LMS-203'. Change the status to In progress .
3	Read the 'Done when' list. If anything is unclear, ask now , not after two days.
4	Get the latest code and make your branch (commands below).
5	Write a small piece, run it, see it work. Repeat. Do not write 500 lines before you run anything.
6	Write the tests. Run all tests. They must all pass.
7	Commit with the task number at the start of the message.
8	Push and open a Pull Request. Fill in the PR template (page C5). Change status to In review and post :mag: in the thread.
9	The lead reviews. Fix every comment and push again. Reply 'done' on each comment.
10	After the merge: delete your branch, set status to Done , write how many hours it really took, and post a screenshot.
11	Go back to step 1.

The Git commands (copy and use)

```
git checkout develop          # go to the shared code
git pull                     # get the newest version
git checkout -b feature/LMS-203-course-crud # make your own branch

# ... write code, run it, write tests ...

python manage.py test        # or: pytest -> everything must pass
git add .
git commit -m "LMS-203: add course create view"
git push -u origin feature/LMS-203-course-crud
# now open the Pull Request on GitHub, base branch = develop

# If develop changed while you worked:
git checkout develop && git pull
git checkout feature/LMS-203-course-crud
git merge develop            # fix any conflicts, run tests again
```

Do and Do not

Do	Do not
Keep each task small. If a task takes more than 1.5 times the estimate, tell the lead.	Do not hide problems. Being late is fine. Being silent is not.
Ask before you change a model that another person owns.	Do not edit a migration that is already merged. Make a new one.
Use URL names like {% url 'courses:detail' slug=c.slug %}.	Do not type web addresses by hand inside templates.
Put business rules in services.py.	Do not put big logic inside templates or views.
Keep secrets in the .env file.	Never commit passwords, keys or the .env file.
Have at most 2 tasks In progress at one time.	Do not start a new task while an old one waits in review; help review a friend's PR instead.

A4. How We Talk to Each Other

Where	Use it for	Reply time
#lms-standup	Your daily message (template below). Before 09:00.	Lead reads by 10:00
#lms-tasks	One thread per task: start, questions, done, screenshot.	Same day
#lms-blockers	You are stuck for 30 minutes or more. Say what you tried.	Lead: within 1 hour
#lms-backend / #lms-frontend	Technical questions. Anyone can answer; do not wait only for the lead.	Same day
#lms-qa	Bugs and test results.	Same day
#lms-github	Automatic PR and test messages. Do not chat here.	-

Daily standup message (copy this)

```
Yesterday: finished LMS-201 (Course model), PR is open
Today:      start LMS-203 (course create view)
Blockers:   none      <- or write exactly what you are waiting for and from whom
Mood:       green / amber / red  (how do you feel about your workload?)
```

How to ask a good question

- Say **what you wanted** to do, **what happened**, and **what you already tried**.
- Paste the error text (not a photo of the screen when text is possible).
- Name the task number and the branch.
- Example: 'LMS-203: I want to show Create only to facilitators with the right. I get a 403 for admin too. I tried adding the role to the mixin. My branch is feature/LMS-203-course-crud. See error below.'

Nobody will be blamed for asking. The lead will only be worried when someone is silent. Asking early is a sign of a good developer.

B1. The Team and Who Owns What

You are the **senior developer and lead**. The other five people take the roles below. Give the strongest backend developer the role **BE-1**, because BE-1 builds the most important base (user model, roles, security). Write real names in the last column.

Tag	Job in simple words	Owns these apps	Hours	Name
PM	Lead (you) . Plans, sets up repo and Slack, reviews every PR, removes blockers, runs the Friday demo, deploys. Also pairs with BE-1 on the hardest first tasks.	All (as reviewer)	9 + reviews	You
BE-1	Foundation and security . Project setup, custom user model, roles and permission mixins, unit pages, approval logic, feedback, attendance. Needs the strongest Django skills.	accounts, attendance	62	_____
BE-2	Courses and go-live . Admin user pages, course create/edit/delete, applications, test CRUD, dashboards, deployment.	courses, enrollments	60	_____
BE-3	Login, data and reports . Database setup, login and register, dashboard redirect, models for applications, marks entry, attendance reports.	assessments, core	60	_____
FE	How it looks . Base layout, navbar, home page, course pages, student pages, mobile checks. Works with plain HTML prototypes early.	templates/, static/	60	_____
QA	Quality . Test setup, test cases, permission tests for every view, bug reports, final user testing with real people.	tests/	48	_____

Hours are the total estimated hours for the whole project (about 299 hours in total). At 15 hours per week each, that is about 4 weeks of work per person spread over 11 weeks, so there is room for learning, reviews and fixing bugs.

If you have to change the roles

- If your team is **weak in Django**: keep BE-1 as the strongest and pair the other two backend developers with BE-1 for their first task.
- If someone is stronger at design than backend: make them **FE** and move the small backend tasks to another person.
- If a person is much slower than the plan: take away the last task in their sprint and give it to the person with the least hours that sprint (see the tracker).
- Never give one person more than two 'High' tasks at the same time.

Work by person and sprint (task numbers)

Sprint	PM	BE-1	BE-2	BE-3	FE	QA
Sprint 0: Setup	001, 007 (3h)	002, 004 (4h)	006 (3h)	003 (2h)	005 (4h)	008 (5h)
Sprint 1: Accounts and Roles	-	101, 102, 104 (10h)	105, 106 (10h)	103, 108, 110 (12h)	107, 111 (12h)	109 (6h)
Sprint 2: Courses, Units and Home Page	-	206, 210 (9h)	201, 203 (11h)	202, 204, 205 (7h)	207, 208, 209 (17h)	211 (8h)
Sprint 3: Applications and Enrollment	-	306 (5h)	303 (5h)	301, 302, 305, 307 (12h)	304, 308 (9h)	309 (6h)
Sprint 4: Assessments and Feedback	-	404, 407 (9h)	402, 406 (9h)	401, 403 (12h)	405 (6h)	408 (6h)
Sprint 5: Attendance	-	501, 502, 503 (16h)	507 (4h)	504, 505 (10h)	506 (4h)	508 (5h)
Sprint 6: Dashboards, Polish and Release	609 (6h)	602, 606 (9h)	601, 605, 608 (18h)	603 (5h)	604 (8h)	607 (12h)

Numbers are the task IDs without 'LMS-'. For example 203 means LMS-203. The full description of every task is in the plan PDF and in the tracker.

B2. What Each Person Does First

The critical path: almost everything waits for three tasks: **LMS-101** (User model) then **LMS-102** (profiles) then **LMS-104** (permission mixins). If these are late, everybody waits. So the lead pairs with BE-1 and the rule is: **101, 102 and 104 must be merged by Wednesday of week 2**. BE-1 does them in this order: **101 - 102 - 104**. (The plan lists 103 before 104 for reading order, but 104 is needed first by others.)

Week 1 (Sprint 0): day by day

Day	PM (Lead)	BE-1	BE-2	BE-3	FE	QA
Mon	LMS-001 repo and branches. Invite everyone. 30-min kick-off: read A1 and A3 together.	Install tools. Start LMS-002 (project and settings).	Install tools. Read A2. Clone repo.	Install tools. Install PostgreSQL. Read A2.	Install tools. Study Bootstrap grid and navbar.	Install tools. Read the workflows A to E in the plan.
Tue	LMS-007 Slack channels, GitHub app, task board. Review LMS-002 PR.	Finish LMS-002. Start LMS-004 (create 6 apps).	LMS-006 CI and code checks (after repo exists).	LMS-003 PostgreSQL connection (after 002 merges). Pair with BE-1 if new.	Start LMS-005 base.html with static files.	Start LMS-008 (pytest setup).
Wed	Review PRs. Check that everyone can run the project.	LMS-004 merged. Start reading LMS-101 details with the lead.	Finish 006. Try the Django tutorial parts 1 to 4.	Finish 003. Django tutorial parts 1 to 4.	Continue 005. Navbar with role links (fake roles).	Continue 008. Draft test cases for Workflow A and B.
Thu	Sprint 0 check: everyone can run, test and open the home page.	Prepare for Sprint 1. Draw the User and Profile fields on paper with the lead.	Review a teammate's PR (learning).	Review a teammate's PR (learning).	Finish 005 and open PR.	Finish 008 and open PR.
Fri	Demo: 'Hello LMS' page from the base template. Plan Sprint 1.	Rest and ready.	Ready.	Ready.	Start LMS-111 (static prototype of home page).	Draft test cases for Workflows C, D, E.

Who does not have a numbered task on some days? That is on purpose. Use quiet time to learn, review a friend's work or prepare. Nobody should sit and wait without telling the lead.

Weeks 2 and 3 (Sprint 1): what to do while others finish

Person	Order of work	If you must wait, do this
BE-1	101, then 102, then 104 (with the lead). Merge each one quickly, even if small.	Write the model tests for User and profiles while the PR waits for review.
BE-3	103 (login, register, password reset) as soon as 101 is merged. Then 108 (dashboard redirect), then 110 (demo data).	Write the login and register templates with FE while 101 is being reviewed.
BE-2	105 (admin users) and 106 (grant / revoke right) as soon as 104 is merged.	Draft the Course and Unit models on a private branch. Do not merge before 104. Read the URL tables in the plan.
FE	107 (profile pages), 111 (static prototypes for home, course detail, dashboards).	Prototypes need no Django. They save days in Sprint 2.
QA	109 tests for roles, registration, grant / revoke, after each PR is merged.	Write the test cases for 'anonymous, student, facilitator, facilitator with right, admin' for every planned URL.

B3. Sprint by Sprint: Who Builds What

Sprint 0 - Setup

Week 1 | 8 tasks | 21h

Person	Tasks (in the order to do them)	Hours
PM	LMS-001 Create GitHub repo, branch protection (main/develop), .gitignore, README LMS-007 Create Slack channels, connect GitHub app, build the standup workflow and task board (see Slack section)	3
BE-1	LMS-002 Create Django project 'config' with split settings (base/dev/prod), django-environ, .env.example LMS-004 Create the 6 apps (core, accounts, courses, enrollments, assessments, attendance) and register them	4
BE-2	LMS-006 Add ruff/black, pre-commit and a GitHub Actions workflow that runs tests on every PR	3
BE-3	LMS-003 Set up PostgreSQL (local or docker-compose) and connect Django to it	2
FE	LMS-005 Base template: Bootstrap 5, navbar that changes per role, messages and pagination partials	4
QA	LMS-008 Set up pytest + factory-boy and write the test-case checklist for workflows A to E	5

Sprint 1 - Accounts and Roles

Weeks 2-3 | 11 tasks | 50h

Person	Tasks (in the order to do them)	Hours
BE-1	LMS-101 Custom User model (role, phone, photo) - MUST be done before the first migrate LMS-102 FacilitatorProfile and StudentProfile models + signal that creates the profile automatically LMS-104 Access-control mixins: RoleRequiredMixin, CourseManagerRequiredMixin	10
BE-2	LMS-105 Admin user management: list, create facilitator, edit, activate/deactivate LMS-106 Admin facilitator list with grant / revoke 'can_manage_courses' button	10
BE-3	LMS-103 Student registration, login, logout and password reset (views + templates) LMS-108 Dashboard redirect by role + three placeholder dashboards LMS-110 Management command create_demo_data (admin, facilitators, students, courses)	12
FE	LMS-107 Profile view and edit pages for all roles LMS-111 Static HTML prototypes (home, course detail, dashboards) so pages are designed before the views exist	12
QA	LMS-109 Write tests: registration, login, role mixins, grant/revoke right	6

Sprint 2 - Courses, Units and Home Page

Weeks 4-5 | 11 tasks | 52h

Person	Tasks (in the order to do them)	Hours
BE-1	LMS-206 Unit CRUD nested under a course + assign facilitator to unit LMS-210 'My units' page for facilitators (units I teach)	9
BE-2	LMS-201 Course model, admin registration and migrations LMS-203 Course create / update / delete views with ownership checks	11
BE-3	LMS-202 Unit model with facilitator FK and ordering LMS-204 Course form validation (dates, capacity) and image upload LMS-205 Publish / close course action (status change)	7
FE	LMS-207 Home page: published course cards, search, filter, pagination LMS-208 Course detail page with unit list and smart 'Apply' button state LMS-209 'Manage courses' table page for admin and course managers	17
QA	LMS-211 Tests: who can create / edit / delete, unpublished courses hidden from home	8

Sprint 3 - Applications and Enrollment

Week 6 | 9 tasks | 37h

Person	Tasks (in the order to do them)	Hours
BE-1	LMS-306 Approve / reject with note; service function creates Enrollment	5
BE-2	LMS-303 Apply view + form; rules: window open, capacity, no duplicates	5
BE-3	LMS-301 Application model with unique (student, course) constraint LMS-302 Enrollment model LMS-305 Manage applications list with filters (status, course) LMS-307 Email on decision (console backend in dev, SMTP in prod)	12
FE	LMS-304 'My applications' page with withdraw button LMS-308 Student 'My courses' page + facilitator unit roster	9
QA	LMS-309 Tests: apply rules, approval flow, capacity edge cases	6

Sprint 4 - Assessments and Feedback		Weeks 7-8 8 tasks 42h
Person	Tasks (in the order to do them)	Hours
BE-1	LMS-404 Feedback create / update form linked to a result LMS-407 In-app notification when feedback is added (optional)	9
BE-2	LMS-402 Test CRUD for the unit's facilitator LMS-406 Facilitator 'pending feedback' filter	9
BE-3	LMS-401 Test, TestResult and Feedback models LMS-403 Bulk marks entry page (formset for all enrolled students)	12
FE	LMS-405 Student results list and result detail with feedback	6
QA	LMS-408 Tests: marks validation, feedback permissions, student privacy	6

Sprint 5 - Attendance		Week 9 8 tasks 39h
Person	Tasks (in the order to do them)	Hours
BE-1	LMS-501 ClassSession and AttendanceRecord models LMS-502 Session CRUD for the unit's facilitator LMS-503 Mark-attendance page (status per student, 'mark all present')	16
BE-2	LMS-507 Admin attendance overview across all units	4
BE-3	LMS-504 Unit attendance report: % per student, highlight low attendance LMS-505 Export attendance to CSV / PDF	10
FE	LMS-506 Student 'My attendance' page	4
QA	LMS-508 Tests: only unit facilitator can mark, no duplicate records	5

Sprint 6 - Dashboards, Polish and Release		Weeks 10-11 9 tasks 58h
Person	Tasks (in the order to do them)	Hours
PM	LMS-609 Deploy, smoke test, hand-over documentation and training	6
BE-1	LMS-602 Facilitator dashboard: my units, upcoming sessions, pending feedback LMS-606 Security review using the checklist in this document	9
BE-2	LMS-601 Admin dashboard: totals, pending applications, recent activity LMS-605 Performance pass: select_related / prefetch_related, indexes, pagination LMS-608 Production settings, Unicorn + Nginx / Whitenoise, env vars, DB backups	18
BE-3	LMS-603 Student dashboard: my courses, next sessions, latest results, attendance %	5
FE	LMS-604 UI polish: responsive check, empty states, 403/404/500 pages	8
QA	LMS-607 Full regression + user acceptance testing with real users	12

C1. Your Weekly Rhythm as the Lead

A simple, repeatable routine is the best way to manage. Do the same things on the same days, so the team always knows what to expect.

Day	Time	What you do	Length
Every day	09:30	Read all standup messages. Reply to blockers. Check the tracker for RED or AMBER people.	15 min
Every day	11:00 and 15:00	Two fixed review windows : review open Pull Requests. Promise: every PR gets a first review within 4 working hours.	30 min each
Monday	09:00	Sprint planning (call): show the sprint goal, walk through each person's tasks, ask 'is anything unclear?'	30 min
Wednesday	14:00	Mid-sprint check : open the tracker. Aim for about 40 to 50 percent of tasks Done or In review. Talk to anyone AMBER or RED.	20 min
Thursday	any	Short 1-to-1 chat (10 min each) with anyone who looks slow, quiet or stressed.	10 min each
Friday	15:00	Demo : each person shows what they built on screen. Then retro : what to start, stop and continue.	45 min
Friday	16:00	Update the tracker, plan next sprint, post the weekly summary in #lms-general.	30 min

Total management time: about 6 to 7 hours per week. You can still write code: keep about 10 hours per week for coding and pairing.

Weekly summary message (post every Friday)

```
Sprint 2, week 1 summary
Done:      8 of 11 tasks (73%)      Hours: 31 of 52 estimated
In review: 2      Blocked: 0      Overdue: 1 (LMS-209, new date Monday)
Demo:      home page shows published courses; create and edit course work
Risks:      FE waiting for unit list PR (LMS-206)
Next week:  finish 209, 211 tests, start sprint 3 models
Thanks:     BE-2 for fast fixes on review comments
```

Setting up the tracker (once)

- Open `LMS_Progress_Tracker.xlsx`. On the **Team** sheet type the real names and the project start Monday.
- Upload the file to Google Drive or OneDrive so everyone edits the same copy, and pin the link in #lms-tasks.
- Team members change only Status, Blocked, Actual hours, Review rounds and Notes. You read the **Summary** sheet.
- Ask every person to update the tracker **before the daily standup**, in less than 2 minutes.

C2. How to Check Progress Fairly

Do not ask 'are you busy?'. Look at **five simple signals**. Together they tell you the truth about each person.

#	Signal	Where to look	Good sign	Warning sign
1	Tasks finished against plan	Tracker: Summary, '% tasks done'	Ahead or on the plan for this week	More than one task behind the sprint plan
2	Time: estimate against real hours	Tracker: Estimated h and Actual h	Real hours within 1.5 times the estimate	Always more than 2 times the estimate, or always far below (maybe rushing)
3	Code quality	Pull Request review rounds	Only 0 to 2 rounds; same mistakes do not come back	3 or more rounds; same comment repeated
4	Testing	Do tests pass? Are permission tests written?	Tests come with the PR	PR without tests; broken tests on develop
5	Communication	Standups, blockers channel	Daily standup; asks early	Silent for 2 days; 'almost done' for 3 days

Traffic light (also shown in the tracker)

Colour	Rule in the tracker	What you do
GREEN	No overdue task and no blocked task	Say thank you. Give the next task on time. Keep the pace.
AMBER	1 to 2 overdue tasks, or 1 blocked task	Talk the same day. Ask: what is stopping you? Offer a 30-minute pairing.
RED	3 or more overdue tasks, or 2 or more blocked tasks	Stop and help now. Look at the whole task list. Split, move or cancel tasks. Tell the team what changed.

Personal scorecard (fill in at the end of each sprint)

Question	1 = needs help	3 = good	5 = excellent	S1	S2	S3	S4	S5	S6
Delivery: finished the agreed tasks	Less than half	Most tasks, small delay	All tasks on time						
Quality: code is clear and follows the rules	Many review comments	A few comments	Almost none						
Testing: tests come with the work	No tests	Basic tests	Includes permission and edge cases						
Communication: standup, questions, help	Often silent	Regular	Proactive; helps others						
Learning: gets better each sprint	Same mistakes	Improving	Teaches teammates						

Use the scorecard to **help people grow**, not to punish. Show it to the person in the 1-to-1 chat and agree on one thing to improve next sprint.

C3. Reviewing Pull Requests

Reviews are your most powerful teaching tool. Be kind, be clear and explain **why**. Use three labels in comments: **Must fix** (blocks merge), **Should fix** (please do), **Idea** (optional).

#	Check	Why it matters
1	Does the code do what the task's 'Done when' list says?	The task is not finished until this is true
2	Is the permission checked on the server (mixin or check), not only hidden in the template?	Security. A student must never see another student's data
3	Are lists limited to the right objects (only my units, only my results)?	Prevents data leaks by changing the number in the URL
4	Are there tests, and do they pass on the CI?	Protects us when other people change the code later
5	Migration files included, small and clear? No old migration edited?	Avoids database problems for the whole team
6	Business rules in <code>services.py</code> , views short?	Easy to read and test
7	No repeated database queries inside loops (use <code>select_related</code> / <code>prefetch_related</code>)?	Pages stay fast when there are many students
8	Forms validate the input. Error messages are clear?	Bad data never reaches the database
9	Page works on a phone-sized screen and has an empty-state message?	Most students use phones
10	No secrets, no debug prints, no commented-out code, clear names?	Clean code is cheaper to keep

Good review comment examples

Must fix: This view is missing `CourseOwnerMixin`. Right now any facilitator can edit any course. Please add it and a test where facilitator B tries to edit facilitator A's course (expect 403).

Should fix: Move the capacity check into `services.approve_application()`. Then the view stays small and we can test the rule alone.

Idea: Nice work on the empty state. Could we reuse it in My Applications?

C4. When Someone Is Behind or Stuck

Being behind is normal in a project. The lead's job is to find the cause and remove it. Never start with blame.

Step by step

1	Look at the facts. Open the tracker: which tasks, how many days late, how many hours used?
2	Talk privately, kindly. Ask: 'What is making this hard?' Then listen.
3	Find the cause (table below).
4	Agree one clear next step and a time to check again (for example tomorrow 15:00).
5	Write it down in the task thread so everyone sees the new plan.
6	Follow up at the agreed time. If nothing changes after two follow-ups, change the plan: split, move or reassign the task.

Common causes and what to do

Cause	How it looks	Your action
Task not clear	Many questions, work restarted, wrong result	Rewrite the 'Done when' list together. Add an example screen.
Task too big	Stays In progress for many days	Split into tasks under 8 hours. Merge the first part quickly.
Missing skill	Many review rounds, same mistakes	30-minute pairing. Send one short lesson or video. Give a smaller similar task next.
Waiting for someone	Blocked, quiet	You (the lead) chase the other person or change the order of work.
Too much work	Tired, late replies, low mood in standup	Move a task to someone with fewer hours this sprint.
Personal problem	Sudden change in attendance or mood	Talk kindly and privately. Adjust the load for a short time. Ask what support is possible.
Low effort	Excuses, no updates, tasks repeatedly 'almost done'	Have a clear talk about expectations. Set two small goals with dates. Follow up. Escalate only after this.

When the whole project is late

- Do not add people at the end. It usually makes things slower.
- Cut scope instead. The tracker shows 'Low' priority tasks first: LMS-110, 307, 407, 507. They can move to version 2.
- Tell stakeholders early, with a new date and the reason.
- Keep the security and permission tasks. Never cut those.

C5. Sprint Gates and Templates

A **gate** is a check at the end of a sprint. If the gate is not passed, do not start the next sprint's big tasks. Fix first.

Gate	Must be true	Demo to show
Sprint 0	Everyone runs the project locally with PostgreSQL. CI runs tests on a PR. Slack channels and the tracker are ready.	Live: 'Hello LMS' page from base.html.
Sprint 1	Custom user model works. Student can register and log in. Admin can create a facilitator and switch course rights on and off. Permission tests pass.	Live: log in as admin, grant a right, log in as that facilitator.
Sprint 2	Facilitator with right can create, edit and delete own course and units. Facilitator without right cannot. Published course appears on home page.	Live: create a course, publish it, see it on the home page.
Sprint 3	Student applies once per course. Reviewer approves or rejects. Approved student is enrolled. Capacity rule works.	Live: apply, approve, see 'My courses'.
Sprint 4	Facilitator creates a test, enters marks for all students and writes feedback. Student sees only their own marks and feedback.	Live: try to open another student's result URL and get 403.
Sprint 5	Facilitator creates a session, marks attendance, sees % per student and exports it. Student sees own attendance.	Live: mark attendance, open the report, download the file.
Sprint 6	Three dashboards, security checklist done, all tests pass, real users tested it, deployed with backups.	Live on the real web address with real accounts.

Pull Request template

Title: LMS-203: Course create / edit / delete views

What I did: (2 or 3 short sentences)

How to test: 1) log in as facilitator with right 2) go to /courses/create/ ...

Screenshots: (add one for every page you changed)

Checklist:

☐ Tests added and passing	☐ Migration included (if models changed)
☐ Permissions enforced on server	☐ Works on phone screen
☐ No secrets or debug code	☐ Actual hours written in the tracker

Bug report template (QA posts in #lms-qa)

Bug: Student can open another student's result

Where: /my/results/12/ Task: LMS-405 Severity: HIGH (privacy)

Steps: 1) log in as student A 2) open /my/results/12/ (belongs to student B)

Expected: 403 page

Actual: Result of student B is shown

Screenshot: (attach)

Severity levels

Level	Meaning	Fix within
HIGH	Data leak, cannot log in, cannot apply, data lost	Same day. Stop other work.
MEDIUM	A feature is wrong but there is a way around it	This sprint
LOW	Spelling, colour, small layout problem	When there is time

Friday retro: three questions

- **Start:** what should we begin doing?
- **Stop:** what wastes our time?
- **Continue:** what works well and must stay?
- Write one action from the retro and give it an owner. Check it next Friday.

C6. Your First Week Checklist

Before Monday

- | | |
|-----|---|
| [] | Confirm the assumptions in the plan (section 1.4) with the business owner. |
| [] | Choose who is BE-1, BE-2, BE-3, FE and QA. Write the names in the tracker Team sheet. |
| [] | Create the GitHub repo and the Slack workspace channels. Upload the tracker and pin it. |
| [] | Send the team the two PDFs and ask them to read A1, A2, A3 and A4. |

Monday

- | | |
|-----|---|
| [] | Kick-off call (30 min): explain the goal in your own words, walk through the five golden rules, show the tracker. |
| [] | Give each person their Sprint 0 tasks in the Slack task board. |
| [] | Ask everyone to post their first standup. |

Tuesday to Thursday

- | | |
|-----|---|
| [] | Review every PR within 4 hours. |
| [] | Sit with BE-1 for the User model design (30 to 60 min). |
| [] | Check that every person can run the project and the tests on their own machine. |

Friday

- | | |
|-----|--|
| [] | Run the first demo, even if it is just a blank page with a navbar. |
| [] | Do the first retro. |
| [] | Update the tracker, send the weekly summary and plan Sprint 1. |

Remember: your main job is not to write the most code. It is to make it easy for five people to finish good work. Clear tasks, fast reviews, and honest, kind talks will give you a better result than working late yourself.